

REDUX

Cours Theorique

Session 14 : Introduction a Redux
Comprendre la Gestion d'Etat Globale

Module M203 - Developpement Front-End

ISTA - 2eme Annee DEVOWFS

Annee 2024/2025

Objectifs de la seance (5h) :

- Comprendre le probleme du **Prop Drilling**
- Maitriser la philosophie Redux (flux unidirectionnel)
- Connaitre les **3 principes fondamentaux** de Redux
- Comprendre les concepts : **Store, Actions, Reducers**
- Installer et configurer Redux DevTools
- Realiser un **Counter avec Redux classique**

Duree : 5 heures

Prerequis : useState, useEffect, Props, Composants

Prochaine session : S15 - Redux Toolkit & Immer

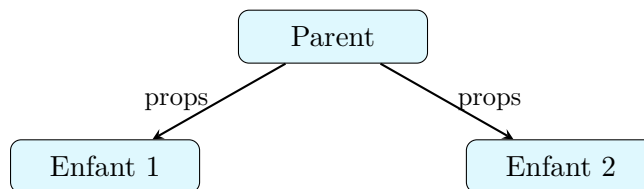
Table des matières

1	Le Probleme : Prop Drilling	2
1.1	Rappel : Comment circulent les donnees en React ?	2
1.2	Qu'est-ce que le Prop Drilling ?	2
1.2.1	Scenario concret : Application E-commerce	2
1.2.2	Code avec Prop Drilling	3
1.3	Problemes du Prop Drilling	3
1.4	La Solution : Un Etat Global	4
2	Qu'est-ce que Redux ?	5
2.1	Definition	5
2.2	Philosophie : Le Flux Unidirectionnel	5
2.3	Les 3 Principes Fondamentaux de Redux	6
3	Les Concepts Cles de Redux	7
3.1	Vue d'ensemble	7
3.2	Action	7
3.3	Action Creator	8
3.4	Reducer	8
3.5	Store	10
4	Installation de Redux	12
4.1	Packages necessaires	12
4.2	Commandes d'installation	12
4.3	Structure de projet recommandee	12
4.4	Redux DevTools	12
5	Guide Pratique : Counter avec Redux Classique	13
6	Recapitulatif	18
6.1	Flux Redux Complet	18
6.2	Checklist Session 14	18

1 Le Probleme : Prop Drilling

1.1 Rappel : Comment circulent les donnees en React ?

En React, les donnees circulent dans **un seul sens** : du composant **parent** vers les composants **enfants** via les **props**.



Exemple simple :

```

1 // Parent passe le nom a l'enfant via props
2 function Parent() {
3   const [user, setUser] = useState({ name: 'Ahmed' });
4   return <Child user={user} />;
5 }
6
7 function Child({ user }) {
8   return <p>Bonjour {user.name}</p>;
9 }
  
```

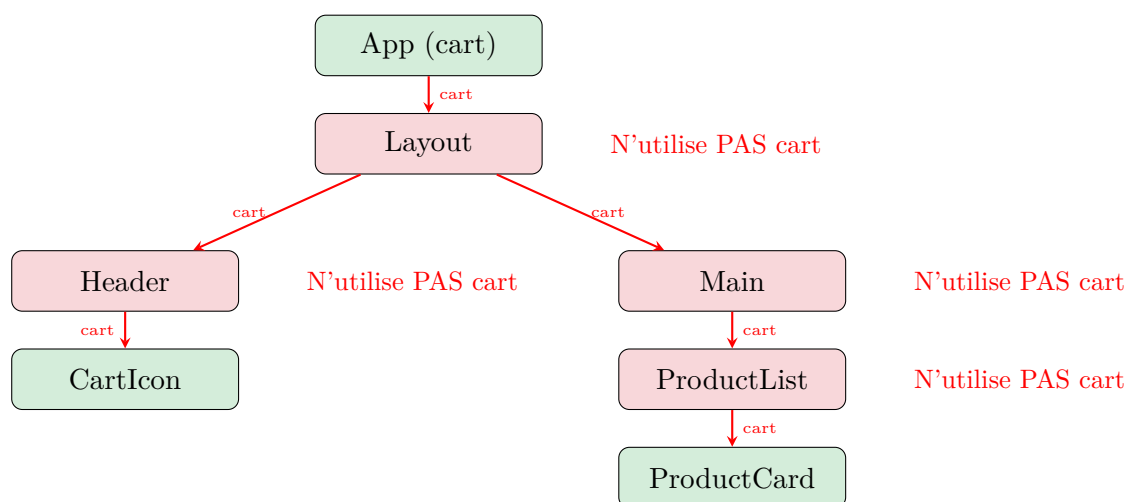
1.2 Qu'est-ce que le Prop Drilling ?

Definition

Prop Drilling (forage de props) : Technique qui consiste a passer des props a travers **plusieurs niveaux** de composants intermediaires, meme s'ils n'utilisent pas ces donnees, juste pour atteindre un composant enfant profond.

1.2.1 Scenario concret : Application E-commerce

Imaginons une application avec un **panier** (cart) qui doit etre accessible depuis plusieurs composants :



Legende : Vert = Utilise cart | Rouge = Ne l'utilise PAS mais doit le passer

1.2.2 Code avec Prop Drilling

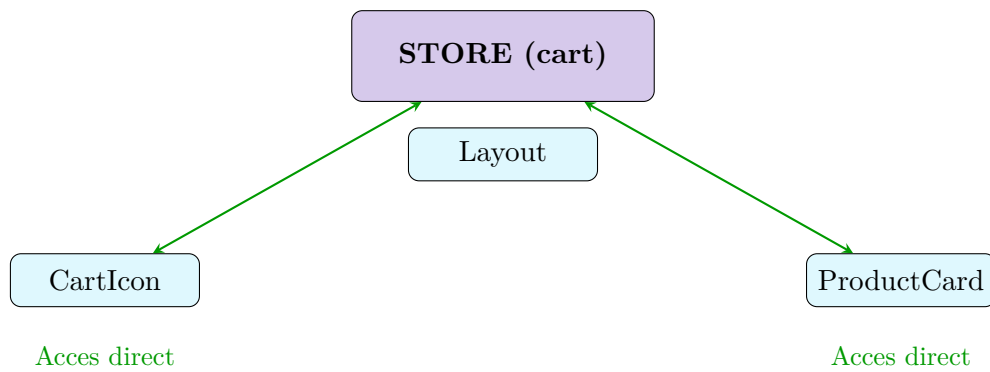
```
1 // App.jsx - Definit le state cart
2 function App() {
3   const [cart, setCart] = useState([]);
4
5   const addToCart = (product) => {
6     setCart([...cart, product]);
7   };
8
9   return (
10     <Layout cart={cart} addToCart={addToCart} /> // Passe a Layout
11   );
12 }
13
14 // Layout.jsx - NE L'UTILISE PAS, juste le passe
15 function Layout({ cart, addToCart }) {
16   return (
17     <div>
18       <Header cart={cart} /> // Passe a Header
19       <Main cart={cart} addToCart={addToCart} /> // Passe a Main
20     </div>
21   );
22 }
23
24 // Header.jsx - NE L'UTILISE PAS, juste le passe
25 function Header({ cart }) {
26   return (
27     <header>
28       <CartIcon cart={cart} /> // Passe a CartIcon
29     </header>
30   );
31 }
32
33 // CartIcon.jsx - UTILISE cart (enfin!)
34 function CartIcon({ cart }) {
35   return <span>Panier ({cart.length})</span>; // Utilise cart.length
36 }
```

1.3 Problemes du Prop Drilling

Attention

1. **Code verbeux** : Beaucoup de repetition pour passer les props
2. **Composants pollues** : Layout, Header recoivent des props qu'ils n'utilisent pas
3. **Maintenance difficile** : Si on ajoute une prop, il faut modifier TOUS les composants intermediaires
4. **Refactoring complexe** : Changer la structure de l'arbre = tout revoir
5. **Lisibilite** : Difficile de suivre d'où viennent les donnees

1.4 La Solution : Un Etat Global



Avec un Store global : Chaque composant accede **directement** aux donnees dont il a besoin, sans passer par les intermediaires.

2 Qu'est-ce que Redux ?

2.1 Definition

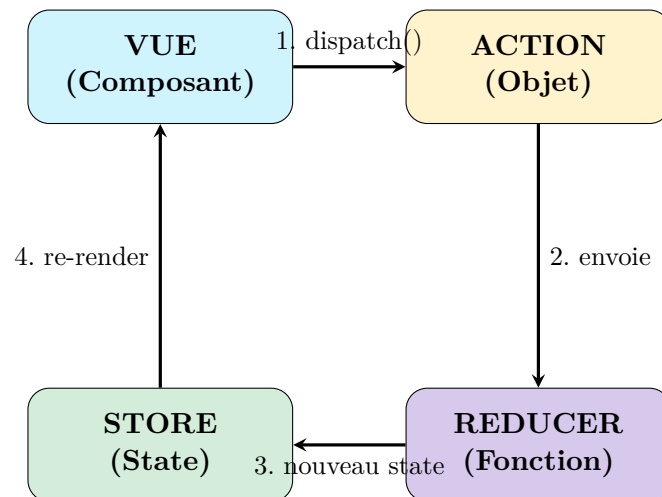
Definition

Redux est une bibliotheque JavaScript de gestion d'etat **predictible** pour applications JavaScript. Elle fournit un **conteneur d'etat centralise** (Store) accessible par tous les composants de l'application.

- Cree en 2015 par **Dan Abramov** et **Andrew Clark**
- Inspire de l'architecture **Flux** de Facebook
- Utilise par des milliers d'applications (Facebook, Instagram, Twitter, Uber...)
- Fonctionne avec React, Angular, Vue, ou meme JavaScript vanilla

2.2 Philosophie : Le Flux Unidirectionnel

Redux suit un **flux de donnees unidirectionnel** (one-way data flow) :



1. L'utilisateur interagit avec la Vue (clic sur un bouton)
2. La Vue **dispatch** une **Action** (envoie un message)
3. Le **Reducer** recoit l'action et calcule le **nouveau state**
4. Le **Store** est mis a jour
5. La Vue se **re-render** avec les nouvelles donnees

2.3 Les 3 Principes Fondamentaux de Redux

Important - A retenir pour l'examen

Ces 3 principes sont **fondamentaux** et souvent demandes aux examens :

1. Source Unique de Verite (Single Source of Truth)

Tout l'etat de l'application est stocke dans un **seul objet JavaScript** appele **Store**.

```
// Un seul store pour toute l'application
const store = {
  user: { name: 'Ahmed', logged: true },
  cart: [{ id: 1, name: 'iPhone' }],
  products: [...],
  ui: { theme: 'dark', language: 'fr' }
};
```

2. L'Etat est en Lecture Seule (State is Read-Only)

La **seule facon** de modifier l'etat est d'emettre une **Action** qui decrit ce qui s'est passe.

```
// INTERDIT : Modifier directement le state
store.cart.push(newItem); // NON !

// CORRECT : Dispatcher une action
dispatch({ type: 'ADD_TO_CART', payload: newItem });
```

3. Les Modifications sont faites par des Fonctions Pures (Pure Reducers)

Les Reducers sont des **fonctions pures** qui prennent l'etat precedent et une action, et retournent un **nouvel etat** (sans muter l'original).

```
// Fonction pure : meme input = meme output
function reducer(state, action) {
  // Retourne un NOUVEL objet, ne modifie pas state
  return { ...state, value: state.value + 1 };
}
```

3 Les Concepts Cles de Redux

3.1 Vue d'ensemble

Concept	Description	Type
Store	Conteneur qui stocke tout le state de l'application	Objet
State	Les donnees de l'application a un instant T	Objet
Action	Message qui decrit ce qui s'est passe	Objet
Action Creator	Fonction qui cree et retourne une action	Fonction
Reducer	Fonction qui calcule le nouveau state	Fonction pure
Dispatch	Methode pour envoyer une action au store	Fonction
Subscribe	S'abonner aux changements du store	Fonction

3.2 Action

Definition

Une **Action** est un objet JavaScript simple qui decrit **ce qui s'est passe**. Elle doit obligatoirement avoir une propriete **type** (string).

Syntaxe

```
1 // Structure d'une Action
2 {
3   type: 'NOM_DE_L_ACTION',    // OBLIGATOIRE - string en MAJUSCULES
4   payload: donnees            // OPTIONNEL - donnees associees
5 }
```

Exemple

```
1 // Action simple (sans payload)
2 const incrementAction = { type: 'INCREMENT' };
3
4 // Action avec payload
5 const addProductAction = {
6   type: 'ADD_PRODUCT',
7   payload: { id: 1, name: 'iPhone', price: 12000 }
8 };
9
10 // Action de suppression
11 const deleteAction = {
12   type: 'DELETE_PRODUCT',
13   payload: 5 // ID du produit a supprimer
14 };
```


3.3 Action Creator

Definition

Un **Action Creator** est une fonction qui **cree et retourne** une action. Cela evite de reecrire l'objet action a chaque fois.

Syntaxe

```
1 // Sans Action Creator (repetitif)
2 dispatch({ type: 'ADD_PRODUCT', payload: { id: 1, name: 'iPhone' }
3   });
4 dispatch({ type: 'ADD_PRODUCT', payload: { id: 2, name: 'MacBook' }
5   });
6
7 // Avec Action Creator (reutilisable)
8 const addProduct = (product) => {
9   return {
10     type: 'ADD_PRODUCT',
11     payload: product
12   };
13 };
14
15 // Utilisation
16 dispatch(addProduct({ id: 1, name: 'iPhone' }));
17 dispatch(addProduct({ id: 2, name: 'MacBook' }));
```

3.4 Reducer

Definition

Un **Reducer** est une **fonction pure** qui prend le state actuel et une action, et retourne un **nouveau state**. Le nom vient de la methode `Array.reduce()`.

Syntaxe

```
1 // Signature d'un Reducer
2 function reducer(state = initialState, action) {
3   switch (action.type) {
4     case 'ACTION_TYPE_1':
5       return nouveauState;
6     case 'ACTION_TYPE_2':
7       return autreNouveauState;
8     default:
9       return state; // IMPORTANT : retourner le state par default
10   }
11 }
```

Attention**Regles strictes pour les Reducers :**

1. **Ne JAMAIS muter** le state existant (`state.value = 5` est INTERDIT)
2. Toujours retourner un **NOUVEL objet** (`return { ...state, value: 5 }`)
3. **Fonction pure** : pas d'effets secondaires (pas d'appels API, pas de `Date.now()`, pas de `Math.random()`)
4. Toujours avoir un case **default** qui retourne le state inchangé

Exemple

Exemple complet : Reducer pour un compteur

```
1 // State initial
2 const initialState = { count: 0 };
3
4 // Reducer
5 function counterReducer(state = initialState, action) {
6   switch (action.type) {
7     case 'INCREMENT':
8       // Retourne un NOUVEL objet avec count + 1
9       return { ...state, count: state.count + 1 };
10
11     case 'DECREMENT':
12       return { ...state, count: state.count - 1 };
13
14     case 'INCREMENT_BY':
15       // Utilise action.payload pour la valeur
16       return { ...state, count: state.count + action.payload };
17
18     case 'RESET':
19       return { ...state, count: 0 };
20
21     default:
22       // IMPORTANT : toujours retourner state si action non reconnue
23       return state;
24   }
25 }
```

3.5 Store

Definition

Le **Store** est l'objet central de Redux. Il contient le state de l'application et fournit des methodes pour interagir avec.

Methodes du Store :

Methode	Description
getState()	Retourne le state actuel du store
dispatch(action)	Envoie une action au reducer pour modifier le state
subscribe(listener)	Enregistre une fonction appelee a chaque changement

Syntaxe

```
1 import { createStore } from 'redux';
2
3 // Creer le store avec le reducer
4 const store = createStore(counterReducer);
5
6 // Lire le state actuel
7 console.log(store.getState()); // { count: 0 }
8
9 // Dispatcher une action
10 store.dispatch({ type: 'INCREMENT' });
11 console.log(store.getState()); // { count: 1 }
12
13 // S'abonner aux changements
14 store.subscribe(() => {
15   console.log('State change:', store.getState());
16 });
```

4 Installation de Redux

4.1 Packages nécessaires

Pour utiliser Redux avec React, nous avons besoin de **deux packages** :

Package	Role
redux	Bibliothèque Redux de base (createStore, combineReducers...)
react-redux	Liaison entre Redux et React (Provider, useSelector, useDispatch)

4.2 Commandes d'installation

```
# Créer un projet React avec Vite
npm create vite@latest mon-app-redux -- --template react
cd mon-app-redux

# Installer Redux et React-Redux
npm install redux react-redux

# Lancer le projet
npm run dev
```

4.3 Structure de projet recommandée

```
src/
  redux/
    actions/
      counterActions.js      # Action creators
    reducers/
      counterReducer.js      # Reducer du compteur
      index.js               # Combine tous les reducers
    store.js                 # Configuration du store
  components/
    Counter.jsx              # Composant UI
  App.jsx
  main.jsx                   # Point d'entrée avec Provider
```

4.4 Redux DevTools

Definition

Redux DevTools est une extension navigateur qui permet de visualiser le state, les actions dispatchées, et de faire du "time travel debugging".

Installation : Rechercher "Redux DevTools" dans les extensions Chrome ou Firefox.

5 Guide Pratique : Counter avec Redux Classique

Nous allons creer un compteur **pas a pas** avec Redux classique (sans Redux Toolkit).

Etape 1 : Creer les Actions

Fichier : src/redux/actions/counterActions.js

```
1 // =====
2 // TYPES D'ACTIONS (constantes)
3 // =====
4 // On definit les types comme constantes pour eviter les fautes de
  frappe
5
6 export const INCREMENT = 'INCREMENT';
7 export const DECREMENT = 'DECREMENT';
8 export const INCREMENT_BY = 'INCREMENT_BY';
9 export const RESET = 'RESET';
10
11 // =====
12 // ACTION CREATORS (fonctions qui creent les actions)
13 // =====
14
15 export const increment = () => {
16   return { type: INCREMENT };
17 };
18
19 export const decrement = () => {
20   return { type: DECREMENT };
21 };
22
23 export const incrementBy = (amount) => {
24   return {
25     type: INCREMENT_BY,
26     payload: amount
27   };
28 };
29
30 export const reset = () => {
31   return { type: RESET };
32 };
```

Etape 2 : Creer le Reducer

Fichier : src/redux/reducers/counterReducer.js

```
1 // Importer les types d'actions
2 import { INCREMENT, DECREMENT, INCREMENT_BY, RESET } from '../
  actions/counterActions';
3
4 // =====
5 // STATE INITIAL
6 // =====
7 const initialState = {
8   count: 0
9 };
10
11 // =====
12 // REDUCER
13 // =====
14 // Fonction pure : (state, action) => newState
15
16 const counterReducer = (state = initialState, action) => {
17   console.log('Action recue:', action.type); // Debug
18
19   switch (action.type) {
20     case INCREMENT:
21       // Retourne un NOUVEL objet (ne modifie pas state)
22       return {
23         ...state,           // Copie les autres proprietes
24         count: state.count + 1
25       };
26
27     case DECREMENT:
28       return {
29         ...state,
30         count: state.count - 1
31       };
32
33     case INCREMENT_BY:
34       return {
35         ...state,
36         count: state.count + action.payload
37       };
38
39     case RESET:
40       return {
41         ...state,
42         count: 0
43       };
44
45     default:
46       // IMPORTANT : retourner state si action non reconnue
47       return state;
48   }
49 };
50
51 export default counterReducer;
```

Etape 3 : Combiner les Reducers (si plusieurs)

Fichier : src/redux/reducers/index.js

```
1 import { combineReducers } from 'redux';
2 import counterReducer from './counterReducer';
3 // import userReducer from './userReducer';
4 // import cartReducer from './cartReducer';
5
6 // =====
7 // ROOT REDUCER
8 // =====
9 // Combine tous les reducers en un seul
10
11 const rootReducer = combineReducers({
12   counter: counterReducer,
13   // user: userReducer,
14   // cart: cartReducer,
15 });
16
17 export default rootReducer;
18
19 // Le state resultant aura cette structure :
20 // {
21 //   counter: { count: 0 },
22 //   user: { ... },
23 //   cart: { ... }
24 // }
```

Etape 4 : Creer le Store

Fichier : src/redux/store.js

```
1 import { createStore } from 'redux';
2 import rootReducer from './reducers';
3
4 // =====
5 // CREATION DU STORE
6 // =====
7
8 // Activer Redux DevTools si disponible
9 const store = createStore(
10   rootReducer,
11   // Active Redux DevTools Extension
12   window.__REDUX_DEVTOOLS_EXTENSION__ && window.
13     __REDUX_DEVTOOLS_EXTENSION__()
14 );
15
16 export default store;
```


Etape 5 : Connecter Redux a React (Provider)**Fichier : src/main.jsx**

```
1 import React from 'react';
2 import ReactDOM from 'react-dom/client';
3 import { Provider } from 'react-redux'; // Import Provider
4 import store from './redux/store';      // Import store
5 import App from './App';
6 import './index.css';
7
8 // =====
9 // PROVIDER
10 // =====
11 // Le Provider rend le store accessible a tous les composants
12
13 ReactDOM.createRoot(document.getElementById('root')).render(
14   <React.StrictMode>
15     <Provider store={store}>
16       <App />
17     </Provider>
18   </React.StrictMode>
19 );
```

Etape 6 : Utiliser Redux dans un Composant

Fichier : src/components/Counter.jsx

```
1 import { useSelector, useDispatch } from 'react-redux';
2 import { increment, decrement, incrementBy, reset } from '../redux/
  actions/counterActions';
3
4 function Counter() {
5   // =====
6   // useSelector : Lire le state du store
7   // =====
8   const count = useSelector((state) => state.counter.count);
9
10  // =====
11  // useDispatch : Obtenir la fonction dispatch
12  // =====
13  const dispatch = useDispatch();
14
15  return (
16    <div style={{ textAlign: 'center', padding: '20px' }}>
17      <h1>Compteur Redux</h1>
18      <h2 style={{ fontSize: '4rem' }}>{count}</h2>
19
20      <button onClick={() => dispatch(increment())}>+1</button>
21      <button onClick={() => dispatch(decrement())}>-1</button>
22      <button onClick={() => dispatch(incrementBy(5))}>+5</button>
23      <button onClick={() => dispatch(reset())}>Reset</button>
24    </div>
25  );
26 }
27
28 export default Counter;
```

Etape 7 : Integrer dans App.jsx

Fichier : src/App.jsx

```
1 import Counter from './components/Counter';
2
3 function App() {
4   return (
5     <div>
6       <Counter />
7     </div>
8   );
9 }
10
11 export default App;
```

6 Recapitulatif

6.1 Flux Redux Complet

1. Utilisateur clique sur +1
- ↓
2. dispatch(increment()) envoie l'action { type: 'INCREMENT' }
- ↓
3. Le counterReducer recoit l'action et calcule le nouveau state
- ↓
4. Le Store est mis a jour : { counter: { count: 1 } }
- ↓
5. useSelector detecte le changement et re-render le composant

6.2 Checklist Session 14

- Comprendre le probleme du **Prop Drilling**
- Connaitre les **3 principes de Redux**
- Savoir creer des **Actions** et **Action Creators**
- Savoir ecrire un **Reducer** (fonction pure)
- Savoir creer et configurer le **Store**
- Utiliser **useSelector** et **useDispatch**
- Installer et utiliser **Redux DevTools**

Prochaine session (S15) : Redux Toolkit & Immer

Nous verrons comment simplifier tout ce code avec **createSlice** et **configureStore** !